

Spark Fuse ComfyUI: User Guide

This guide explains how to render a ComfyUI workflow on a Spark Fuse cloud GPU. It assumes no prior knowledge of the project. By the end you will be able to stage your models once, submit a workflow, watch it run, and download the result, with warm runs completing in about a minute.

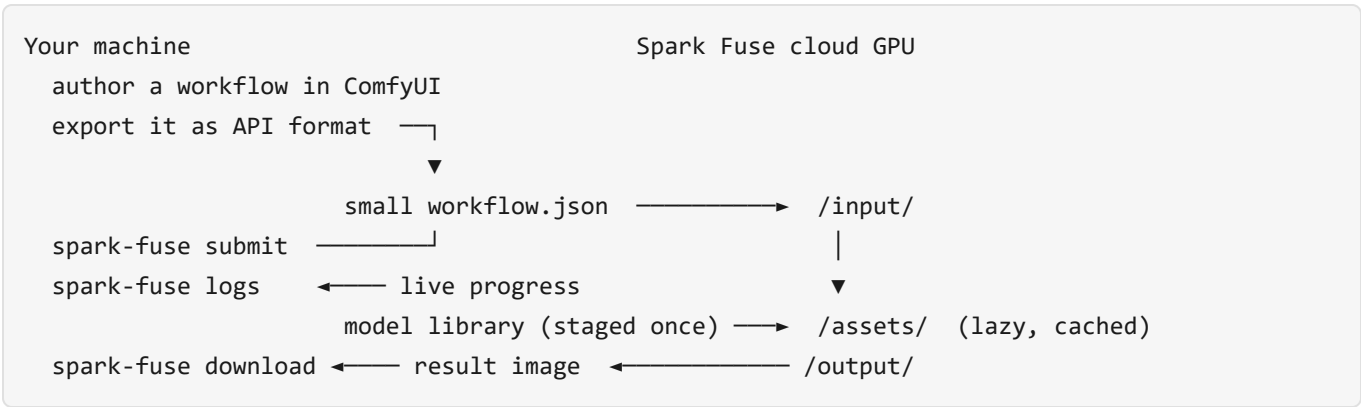
What this is

This project lets you run heavy ComfyUI workflows (Flux and similar) on rented cloud GPUs instead of your own machine. There are four pieces:

- 1. **The image** (this repository): a Docker image that runs ComfyUI headlessly, published at `ghcr.io/vfxguru/spark-fuse-comfyui`. You do not build it; Spark Fuse pulls it automatically.
- 2. **The messenger**: a Python client and command line tool that talks to the Spark Fuse API. See [spark-fuse-messenger](#).
- 3. **ShareSync**: Spark Fuse's cloud storage. Your models and workflow live here, and results come back here.
- 4. **The ComfyUI node** (new): an extension that submits the current workflow from inside ComfyUI with a button. See [spark-fuse-comfyui-node](#). This is the friendliest path and is covered briefly at the end.

Your local ComfyUI and the cloud ComfyUI never talk to each other directly. Your local ComfyUI is only the authoring canvas. Everything moves as files on ShareSync.

How a render flows



The model library is staged on ShareSync once and mounted read-only and lazily at `/assets`. Only the bytes a render actually reads are pulled, and the library is cached on the compute node across jobs. Only

the small `workflow.json` is sent each time. Combined with cached-image affinity, a warm run skips both the image pull and the model copy, which is why repeat renders are fast.

One-time setup

1. A Spark Fuse account and credentials

You need a Spark Fuse account with API access: a host URL, an email, and a password.

2. The messenger client

```
git clone https://github.com/VFXGuru/spark-fuse-messenger
cd spark-fuse-messenger
uv sync
```

Copy `.env.example` to `.env` and fill in your details:

```
SPARK_HOST=https://api.prod.aapse1.sparkcloud.studio
SPARK_EMAIL=you@yourcompany.com
SPARK_PASSWORD=your-password
```

Activate the environment in each new terminal and confirm it works:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
.\.venv\Scripts\Activate.ps1
spark-fuse login
```

3. The Spark desktop app (for staging models)

Install the Spark desktop app from sparkcloud.studio/downloads. It surfaces your ShareSync storage as an ordinary folder in File Explorer (Windows) or Finder (macOS), so you can copy files into it like any local folder.

4. Stage your model library once

In your ShareSync folder, create a folder for your model library and lay it out with the usual ComfyUI subfolders. For example, a folder `comfy-models` containing:

```
comfy-models/  
├─ diffusion_models/      e.g. flux-2-klein-9b-fp8.safetensors  
├─ text_encoders/         e.g. the text encoder your workflow loads  
├─ vae/  
├─ checkpoints/  
├─ loras/  
└─ controlnet/
```

Copy your model files in and let the desktop app finish uploading. You can confirm in the ShareSync web interface that the folder shows its full size. This upload runs at your connection's upload speed and only happens once. Every later job references this same path with no further upload.

The path you pass later is the path as it appears in ShareSync. A folder you see as `comfy-models` at the root of your Personal space is the path `/comfy-models/`.

Render a workflow (command line)

1. Author and export the workflow

Build your workflow in ComfyUI with a Save Image node at the end. Export it in **API format** (the workflow menu's **Export (API)** option, not an ordinary save). Save the file as `workflow.json`. The API format is required because Spark Fuse submits to ComfyUI's programmatic endpoint; an ordinary save is rejected.

2. Stage the workflow

Create a small folder in ShareSync, for example `klein-job`, and copy only `workflow.json` into it. This is the per-render input. The models do not go here; they are served from `/assets`.

3. Choose a GPU and check the cost

```
spark-fuse skus  
spark-fuse estimate g6.xlarge
```

`skus` lists every GPU type with its memory. `estimate` shows the hourly rate. Pick the smallest GPU that fits your models to keep cost down, or a larger one for the fastest single render. As a guide, a model set of around 17 GB runs comfortably on any 24 GB card.

4. Submit

Point `--input-path` at the small workflow folder and `--assets-path` at your model library, and pass `MODEL_BASE_DIR=/assets` so ComfyUI reads models from the assets mount. Pin the image by digest and

ask for cached-image affinity:

```
spark-fuse submit --image ghcr.io/vfxguru/spark-fuse-comfyui@sha256:<digest> --command  
python3.13 --command /runner/spark_fuse_run.py --instance-type g6.xlarge --input-path "/klein-  
job/" --assets-path "/comfy-models/" --env MODEL_BASE_DIR=/assets --image-affinity required
```

It prints a job ID. If you get an immediate HTTP 400 about the input path, the folder has not finished syncing to ShareSync yet; confirm it in the web interface, then resubmit. No GPU is billed in that case.

5. Watch the logs and download

```
spark-fuse logs <job-id>  
spark-fuse download <job-id> .\results
```

Connect to the logs straight away (there is no replay). When the job succeeds, the result image is downloaded into `.\results`.

What to expect on cold and warm runs

The first run on a freshly published image is a cold start: the image is pulled and the model bytes are read for the first time. Once a node has run your job, it caches both the image and the assets. With `--image-affinity required`, later runs are steered onto that warm node, skip the image pull, read models from local disk, and finish in roughly a minute. You can see whether a run hit the cache with `spark-fuse status <job-id>`, which now reports the image cache result.

Render a workflow (ComfyUI node)

For a one-click experience from inside ComfyUI, install the [spark-fuse-comfyui-node](#) extension. It adds a button that ships the current workflow to Spark Fuse, shows live progress, and loads the finished image back into ComfyUI, using the same assets mount and affinity settings described above. Installation and configuration are covered in that repository's README. This node is new; this section will be expanded here once it has been through field testing.

Troubleshooting

The runner reports a clear exit code, shown in the job summary:

Exit code	Meaning	What to do
0	Success	Download your output.
1	Workflow executed but ended in error	Read the ComfyUI error in the log; usually a bad node setting.
2	Bad input	<code>workflow.json</code> is missing, unreadable, or not API format; or a model path is wrong.
3	ComfyUI rejected the workflow	The log names the missing node type; that custom node is not in the image.
4	Job timed out	Increase the timeout, or check why the render stalled.
5	Completed but produced no files	Your workflow has no save node, so nothing was written to <code>/output</code> .
6	ComfyUI failed to start or died	Check the log near startup; report it if it persists.

Other common situations:

- **Submission fails with HTTP 400 about the input or assets path.** The folder has not finished syncing to ShareSync, or the path is misspelt. Confirm it in the ShareSync web interface, then resubmit.
- **Submission fails with `account_check_failed` or an estimate returns a pricing error.** That GPU type has no price configured. Choose a different instance type or ask Spark Cloud Studio to add the pricing row.
- **The workflow is rejected as the wrong format.** Re-export it using **Export (API)**, not an ordinary save.
- **A model is not found.** Check that the file sits under the correct subfolder of your assets library and that the name in the workflow matches exactly.
- **The first run is slow.** That is the expected cold start. Run it again and, with image affinity, the warm run is much faster.

License

MIT, Copyright (c) 2026 VFXGuru.